

Technical Report

Personalized Movie Recommendation System

Prepared for: Cult Finance Club — Open Projects 2026

Institution: IIT Roorkee | Author: Anubhav Kumar | Date: June 2026

1. Problem Understanding & Motivation

If you've ever spent ten minutes scrolling through a streaming platform only to give up and rewatch something familiar, you've experienced the problem this project tries to solve. Streaming services carry thousands of titles, and the sheer volume of choice can be more paralyzing than helpful. Recommendation systems exist precisely to cut through that noise — they act as intelligent filters that learn what a particular user is likely to enjoy and surface it before the user even thinks to search.

This project takes inspiration from the Netflix Prize challenge, one of the most influential open competitions in applied machine learning. Our goal is to build and evaluate algorithms that tackle two related but distinct problems:

- **Rating Prediction** — estimating the numerical score a user would assign to a movie they haven't seen yet. We measure accuracy here using Root Mean Squared Error (RMSE), which penalizes large prediction errors more heavily.
- **Ranking & Recommendation** — generating a ranked shortlist of movies a user is genuinely likely to enjoy. We evaluate this with a custom Mean Average Precision at 10 (MAP@10) metric, which rewards putting the best recommendations near the top of the list.

We implement and compare two well-established algorithmic families using the scikit-surprise framework: Singular Value Decomposition (SVD), a model-based matrix factorization approach, and User-Based Collaborative Filtering (KNNWithMeans), a memory-based neighborhood method. The contrast between these two is instructive — one compresses user behavior into a compact learned representation, while the other reasons directly from observed similarities between users.

2. Exploratory Data Analysis (EDA)

Before training anything, it's worth spending time understanding the data itself. Rushing straight to modeling often leads to surprises later — EDA is where you build intuition about what the numbers actually represent.

We worked with a filtered subset of 1 million ratings drawn from the MovieLens dataset, formatted to match the Netflix Prize raw data schema. The dataset spans 6,040 users and 2,514 movies. One number that immediately stands out: the user-movie interaction matrix is 93.56% sparse. The vast majority of possible user-movie pairs have no observed rating at all — which is entirely realistic for streaming, but it does make the modeling problem significantly harder.

2.1 Rating Distribution & Optimistic Bias

Ratings run on a 1–5 star scale, with a mean of 3.59. The distribution is noticeably right-skewed: the 3-star and 4-star buckets alone account for roughly 60% of all ratings.

This skew has a name — optimistic bias — and it's not accidental. Users don't rate randomly. They tend to rate movies they actively chose to watch, and movies they chose to watch are already pre-filtered by their own preferences. A naive model that always predicts 4 stars will achieve a deceptively low RMSE simply because most ratings cluster near that value. Any model worth deploying must do meaningfully better than this baseline by actually capturing individual taste, not just the population average.

2.2 User Activity and Movie Popularity (Power Law)

Both user activity and movie popularity follow a classic power-law distribution — a small number of highly active users account for a disproportionate share of total ratings, and a small set of blockbuster titles dominates the interaction matrix. Concretely, the top 100 most popular movies account for nearly 19% of all ratings.

This has practical consequences. Neighborhood-based models are susceptible to popularity bias: because popular movies have dense rating columns, they end up being over-recommended, while long-tail niche films — often exactly what makes a recommendation system valuable — get systematically buried. Awareness of this dynamic shaped several of our design decisions downstream.

3. Methodology & Mathematical Formulations

3.1 Model 1: Singular Value Decomposition (SVD)

SVD is a latent-factor matrix factorization model. Rather than reasoning directly about which movies a user has rated, it learns to project both users and movies into a shared latent space of dimension f . A rating prediction is modeled as the inner product of the user and item latent vectors, adjusted for global, user-level, and item-level biases:

$$r_{u,i} = \mu + b_u + b_i + p_u^T q_i$$

Where μ is the global mean rating, b_u captures user u 's personal tendency to rate above or below average, b_i captures movie i 's average reception relative to the global mean, p_u is user u 's latent preference vector, and q_i is movie i 's latent characteristic vector.

Optimization via Stochastic Gradient Descent (SGD)

The model is trained by minimizing a regularized squared error over all observed ratings R :

$$\min \sum (r_{u,i} - r_{u,i})^2 + \lambda (p_u^2 + q_i^2 + b_u^2 + b_i^2)$$

The λ term is an L2 regularization penalty that keeps parameters from growing arbitrarily large — without it, the model would overfit the training data quickly. Parameters are updated via SGD on each observed rating, nudging the latent vectors and biases in the direction that reduces prediction error while keeping them regularized.

3.2 Model 2: User-Based Collaborative Filtering (KNNWithMeans)

While SVD compresses behavior into latent vectors, User-Based Collaborative Filtering takes a more direct approach: to predict how user u would rate movie i , it finds the k most similar users who have already rated that movie and aggregates their ratings. KNNWithMeans adjusts for the fact that different users use different parts of the rating scale by centering around each user's personal mean:

$$r(u,i) = \mu_u + [\sum \text{sim}(u,v) \cdot (r_{v,i} - \mu_v)] / \sum |\text{sim}(u,v)|$$

Similarity between users is computed via Cosine Similarity over the set of movies both users have rated. A user pair with a high cosine similarity tends to agree on the movies they've both seen, making their ratings informative predictors for unseen items.

4. Hyperparameter Configurations

Both models were trained on an 80/20 train/test split. The hyperparameters below were chosen to balance expressiveness with regularization — enough capacity to capture real patterns, but not so much that the models memorize noise.

SVD

- Latent Factors (f): 100
- Training Epochs: 20
- Learning Rate (γ): 0.005
- Regularization (λ): 0.02
- Random Seed: 42 (for reproducibility)

User-Based KNN

- Maximum Neighbors (k): 30
- Similarity Metric: Cosine
- Mode: User-based collaborative filtering

The SVD configuration follows established defaults from the original Netflix Prize literature, while the KNN neighbor count of 30 was chosen as a practical balance between signal quality and computational cost.

5. Experimental Results & Model Evaluation

The results were unambiguous: SVD outperformed User-Based Collaborative Filtering across every dimension we measured. The table below summarizes the comparison:

Metric	SVD (Matrix Factorization)	User-Based KNN
Training Time	2.01s (SGD updates)	220.55s (pairwise matrix build)
Prediction Speed	1.29s (dot products)	93.50s (live neighbor lookup)
Memory Usage	Low — stores factor matrices only	High — $O(U^2)$ similarity matrix

Metric	SVD (Matrix Factorization)	User-Based KNN
Handles Sparsity	Very well (latent space projection)	Poorly (fails on low user overlap)
Explainability	Low (abstract latent dims)	High (direct user similarity)
RMSE ↓	0.9307	0.9838
MAP@10 ↑	0.7359	0.7067

Technical Analysis

Accuracy. SVD's lower RMSE (0.9307 vs 0.9838) reflects a fundamental advantage: latent space projection handles sparse data gracefully. When two users have rated very few movies in common, KNN has almost nothing to work with. SVD sidesteps this by learning a compressed representation of preferences that generalizes across the entire dataset.

Computational performance. The gap in training and prediction time is striking — KNN takes over 100× longer for both. This is a consequence of its lazy learning architecture: KNN defers all computation to query time, requiring a traversal of the full user similarity space for every single prediction. At scale, this becomes completely impractical. SVD's inference is a simple dot product, which is fast regardless of dataset size.

Explainability tradeoff. It's worth noting that KNN does have one genuine advantage: its recommendations are interpretable. Telling a user 'this was recommended because four people with very similar taste to yours gave it four stars' is meaningful in a way that 'your latent vector has high affinity for dimension 37' is not. For production systems where user trust matters, this is a real consideration.

6. Success & Failure Case Studies

6.1 Success: Genre Preference Capturing (Active User)

An active user who consistently rates Sci-Fi films highly (Star Wars, Star Trek, etc.) gets projected into a region of the latent space dominated by Sci-Fi factors. SVD's top-10 recommendations for this user are almost entirely high-action Sci-Fi and fantasy titles — without any explicit genre metadata being used. The model infers genre affinity purely from co-rating patterns, which is exactly what latent factor models are designed to do.

6.2 Success: User Similarity Consistency

We identified two users with high cosine similarity in the KNN model and compared their SVD recommendations. Six out of ten recommended movies were identical. This validates that SVD's learned latent representations align well with the direct user-similarity structure captured by KNN — the two methods converge on similar answers through very different mathematical paths.

6.3 Failure: Cold-Start Limitation

A new user who has rated only one movie presents a difficult problem. With almost no history to work with, their latent vector p_u remains close to the origin. The model falls back on item biases b_i ,

effectively recommending whatever movies have the highest average ratings in the training set. The result is a generic, popularity-driven list that feels nothing like personalization. This is one of the most well-known failure modes of collaborative filtering.

6.4 Failure: Popularity Bias

Evaluating SVD recommendations across 20 randomly selected users reveals that a large share of recommended movies come from the top 10% most popular items. This is a structural bias, not a model bug. Collaborative filtering naturally gravitates toward popular items because they have dense columns in the interaction matrix, generating stronger signals. Niche content with few ratings gets systematically underweighted, reducing the diversity and surprise value of recommendations.

6.5 Explainability via KNN

One thing neighborhood models do well is generate human-readable justifications. For any given recommendation, we can locate the user's nearest neighbors who rated that movie highly and produce an explanation like: "Recommended because 4 users with very similar taste to yours rated Star Wars 4 stars or higher." This kind of transparency can meaningfully increase user trust and perceived fairness — something worth preserving even as we move toward more powerful models.

7. Engineering Solutions & Innovation

7.1 Cold-Start Strategy: Online Latent User Projection

The cold-start problem is solvable without retraining. We built an Interactive Onboarding Quiz into the dashboard: new users rate a handful of popular movies, and we immediately project those ratings into the SVD latent space in real-time using Ridge Regression.

The key insight is that we don't need to retrain anything. The movie latent vectors q_i and item biases b_i learned during training are fixed. We only need to estimate the new user's bias b_u and latent vector p_u , which can be done analytically in milliseconds from even a small number of ratings:

- First, estimate the user bias: $b_u = \text{average of } (r_{u,i} - \mu - b_i) \text{ across rated movies.}$
- Then, solve for the latent vector using L2-regularized least squares over the rated item vectors.

This allows the system to start personalizing from the very first interaction, turning what would have been a cold-start failure into a reasonably warm start.

7.2 Hybrid SVD + Genre Content Recommendation Engine

To reduce popularity bias and improve personalization, we built a content-hybrid scoring system that blends the SVD collaborative signal with explicit genre matching. Each movie has a genre vector m_i , and each user's genre profile u_g is constructed from their highly-rated movies (rating ≥ 4):

$$u_g = \sum (r_{u,j} - 3.0) \cdot m_j \text{ for } j \text{ in user's positively-rated movies}$$

Both vectors are normalized to unit length, and the genre similarity score is their dot product. The final hybrid score blends the two signals:

$$S_{\text{hybrid}} = \alpha \cdot S_{\text{SVD}} + (1 - \alpha) \cdot (1 + 4 \cdot \text{sim_genre})$$

We set $\alpha = 0.7$, giving 70% weight to the collaborative SVD signal and 30% to genre content matching. This balance was chosen empirically: too much genre weight oversimplifies user

preferences; too little fails to provide the diversity boost we were after.

8. Conclusions & Future Work

The comparison between SVD and User-Based KNN confirms what the recommender systems literature has long suggested: model-based matrix factorization is simply a better fit for large-scale, sparse datasets. SVD delivered better rating prediction accuracy (RMSE = 0.9307 vs 0.9838), higher ranking quality (MAP@10 = 0.7359 vs 0.7067), dramatically faster training and inference, and lower memory overhead — while KNN's only clear advantage was explainability.

Beyond model selection, two of the more interesting contributions in this work are the cold-start projection strategy and the hybrid recommendation engine. The former makes personalization possible from the first user interaction without any model retraining; the latter provides a practical lever for tuning the tradeoff between collaborative accuracy and content-based diversity.

What's Next

- **Neural Collaborative Filtering (NCF):** Matrix factorization assumes the user-item relationship is a simple inner product — a linear interaction. Neural architectures can model non-linear interactions, potentially capturing preference patterns that SVD misses entirely.
- **Diversity & Novelty Adjustments:** Incorporating an explicit popularity penalty into the ranking function would promote long-tail content and counter the feedback loops that keep popular movies over-recommended. This is where recommendation systems can genuinely surface things users wouldn't have found on their own.